

Contents

Introduction	1
1 Project Overview	1
2 Project Objectives	1
3 Core Game Features	1
Design and Implementation	3
4 System Architecture	3
5 Technical Implementation Details	3
5.1 Graphics Rendering	3
5.2 Game Loop and Timing	4
5.3 Movement and Boundaries	4
5.4 Collision Detection	4
5.5 Input Handling	5
5.6 State Management	5
Game Screenshots	6
Code Highlights	8
6 Main Game Loop and State Update (<code>main.cpp</code>)	8
7 Car Class (<code>Car.h</code> , <code>Car.cpp</code>)	9
8 Level Class (<code>Level.h</code> , <code>Level.cpp</code>)	13
9 Texture Handling (<code>Texture.h</code> , <code>Texture.cpp</code>)	17
Key Gameplay Mechanics	20
10 Dynamic Road System	20
11 Obstacle and Enemy System	22
12 Scoring, Progression, and High Score	23
Challenges and Solutions	26
13 Performance Optimization	26
14 Collision Detection Accuracy and Consistency	26
15 Visual Cohesion (Scrolling Synchronization)	27
Conclusion and Future Work	28
16 Project Summary	28
17 Learning Outcomes	28
18 Future Enhancements	29
References	30
Appendix	31
19 Project Directory Structure	31
20 Complete Source Code Access	31
21 Build and Run Instructions	32

Abstract

NeedForSpeedInfinity is a fast-paced, arcade-style racing game developed using C++ and the OpenGL graphics library, leveraging FreeGLUT for windowing and GLEW for extension management. This project, undertaken for the Computer Graphics course at Baku Engineering University, simulates a high-speed driving experience where the player navigates an endlessly scrolling road, avoiding dynamically generated obstacles and enemy vehicles. Key features include progressive difficulty scaling based on score and time, collision detection, projectile mechanics, persistent high score tracking, and distinct game states (Menu, Playing, Paused, Game Over). The implementation focuses on core graphics concepts such as 2D rendering within an orthographic projection, texture mapping using the stb_image library, state management, and event handling. This report details the project's objectives, design choices, implementation specifics, challenges encountered, and potential future enhancements.

Introduction

1. Project Overview

NeedForSpeedInfinity is a dynamic, arcade-style racing game developed in C++ utilizing the OpenGL graphics API. Created as a project for 2nd Inter Semester Activity (*SDF-2*) for the Computer Graphics course at Baku Engineering University, it offers a fast-paced driving experience where players must skillfully maneuver a car along a scrolling road, avoiding obstacles and contending with enemy vehicles. The game employs FreeGLUT for window management and input handling, GLEW for accessing OpenGL extensions, and the `stb_image` library for texture loading. Its core design revolves around an endless runner concept with increasing speed and complexity.

2. Project Objectives

The primary goals set forth for the development of NeedForSpeedInfinity were:

- To gain practical experience with OpenGL fixed-function pipeline rendering techniques (vertices, colors, textures).
- To implement a smooth, continuous scrolling environment simulating forward motion.
- To develop robust collision detection between the player, obstacles, and enemies.
- To design and implement dynamic spawning systems for obstacles and enemy vehicles.
- To create a scoring mechanism that rewards survival time and skillful play (e.g., destroying enemies).
- To incorporate game state management (Menu, Play, Pause, Game Over).
- To implement persistent storage for high scores.
- To structure the codebase using object-oriented principles for maintainability and clarity.

3. Core Game Features

The final version of NeedForSpeedInfinity includes the following key features:

- ✓ **Endless Scrolling Road:** A procedurally textured road scrolls vertically, creating the illusion of high-speed travel, complete with synchronized lane markings.
- ✓ **Dynamic Obstacles:** Multiple types of obstacles (Barriers, Cones, Rocks, Trees) are randomly generated and positioned within the driving lanes.
- ✓ **Enemy Vehicles:** AI-controlled cars spawn and travel down the road at increased speeds, adding a significant challenge.

- ✓ **Player Controls:** Responsive controls (Keyboard/Arrow Keys) for horizontal and vertical movement within defined road boundaries.
- ✓ **Projectile System:** The player car can fire projectiles to destroy obstacles and enemy cars.
- ✓ **Collision Detection:** Axis-Aligned Bounding Box (AABB) checks detect collisions between the player, projectiles, obstacles, and enemies.
- ✓ **Health and Damage:** The player car has a health system, taking damage upon colliding with obstacles. Collision with an enemy car results in immediate game over.
- ✓ **Scoring System:** Points are awarded based on survival time (scaled by speed), destroying obstacles, and destroying enemy cars.
- ✓ **Progressive Difficulty:** The game's speed and the spawn rate of obstacles/enemies increase over time and with the player's score.
- ✓ **High Score Persistence:** The highest achieved score is saved to a file ([high-score.txt](#)) and loaded upon startup.
- ✓ **Game States:** Clear transitions between Main Menu, Gameplay, Paused, and Game Over states.
- ✓ **HUD:** An in-game Heads-Up Display shows the current score, high score, player health, and current level.

Design and Implementation

4. System Architecture

NeedForSpeedInfinity adopts an object-oriented design to encapsulate different functionalities and promote code organization. The primary components are:

- ◇ **main.cpp (Core Engine):** Initializes OpenGL, GLUT, and GLEW. Manages the main game loop (`update`, `display`), handles game states (`GameState` enum), processes user input (keyboard/special keys), sets up the projection and viewport, and orchestrates rendering based on the current state.
- ◇ **Car Class (Car.h, Car.cpp):** Represents both the player vehicle and enemy vehicles. Encapsulates properties like position, speed, health, dimensions, and state (active/inactive). Manages movement logic (including boundary checks), rendering (using textures), projectile firing and updating, and collision detection methods (`checkCollision`, `checkProjectileCollision`). Uses a static member for the shared enemy car texture.
- ◇ **Level Class (Level.h, Level.cpp):** Manages the game world and progression. Responsible for rendering the scrolling road and lane lines, spawning/updating/rendering obstacles and enemy cars (`std::vector<Obstacle>`, `std::vector<Car>`), calculating and updating the score and level, managing the road scroll speed, handling high score persistence, and orchestrating collision checks via the `Car` class methods.
- ◇ **Texture Class (Texture.h, Texture.cpp):** A utility class that encapsulates OpenGL texture loading (using `stb_image.h`), binding, and unbinding. Simplifies the process of applying textures to game objects.
- ◇ **GameState Enum (main.cpp):** Defines the possible states of the game (MENU, PLAYING, PAUSED, GAME_OVER), controlling the flow and what is rendered or updated.

This modular architecture facilitates separation of concerns, making the game logic easier to understand, debug, and extend.

5. Technical Implementation Details

5.1. Graphics Rendering

The game utilizes the legacy fixed-function pipeline of OpenGL for rendering.

- **Setup:** GLEW is initialized to access necessary OpenGL functions. Depth testing, texture mapping (`GL_TEXTURE_2D`), and alpha blending (`GL_BLEND`) are enabled.
- **Projection:** An orthographic projection (`gluOrtho2D` or `glOrtho`) is used, effectively creating a 2D perspective suitable for this style of game. Aspect ratio

correction is applied in the `reshape` function to handle different window sizes or full-screen mode.

- **Drawing:** Objects like the car, obstacles, and road segments are drawn as textured quads using `glBegin(GL_QUADS)`, `glTexCoord2f`, and `glVertex3f`. Colors are set using `glColor3f` or `glColor4f`.
- **Texturing:** The `Texture` class loads images (primarily PNGs) using `stb_image.h`, generates OpenGL texture objects (`glGenTextures`, `glBindTexture`, `glTexImage2D`), and sets parameters like filtering (`GL_LINEAR`) and wrapping (`GL_REPEAT`). Alpha blending allows for transparency effects, crucial for non-rectangular sprites if using RGBA textures.

5.2. Game Loop and Timing

- **Loop:** `glutTimerFunc` establishes a callback (`update`) that runs periodically (targeted for 60 FPS). `glutPostRedisplay` is called within `update` to trigger a redraw via the `display` function.
- **Delta Time:** A time difference (`deltaTime`) is calculated each frame based on `glutGet(GLUT_ELAPSED_TIME)`. This ensures that movement and game logic proceed consistently regardless of minor fluctuations in frame rate. `deltaTime` is passed to the `update` methods of `Car` and `Level`.

5.3. Movement and Boundaries

- **Player:** The `Car` class updates its `x` and `y` position based on `horizontalSpeed`, `verticalSpeed`, and `deltaTime`. Input functions (`moveLeft`, `moveRight`, etc.) set these speeds, while key-up events stop the movement (`stopHorizontal`, `stopVertical`).
- **Boundary Enforcement:** Crucially, the `Car::update` method checks the *potential* new position (`newX`, `newY`) against calculated boundaries before applying the movement. These boundaries are derived from the road width and lane line positions (defined in `Level`), including safety margins, to prevent the car from visually crossing the yellow lines. If a boundary would be crossed, the car's position is clamped to the boundary, and its corresponding speed is set to zero.
- **Scrolling:** The `Level` class updates `roadOffset` based on `scrollSpeed` and `deltaTime`, which is then used in texture coordinate calculations for the road (`glTexCoord2f`) to create the scrolling effect. Obstacles and enemies simply have their `y` coordinate decreased each frame based on `scrollSpeed` (or a multiple of it for enemies) and `deltaTime`.

5.4. Collision Detection

- **Method:** Simple Axis-Aligned Bounding Box (AABB) intersection tests are used.
- **Implementation:** The `Car::checkCollision` method takes the position and dimensions of another object (obstacle or enemy car) and checks if its own bounding

box (calculated using its `x`, `y`, and effective width/height) overlaps with the other object's box.

- **Projectile Collisions:** `Car::checkProjectileCollision` iterates through the car's active projectiles, performing an AABB check between each projectile (using a fixed size) and the target object (obstacle/enemy). If a hit occurs, the projectile's `isActive` flag is set to `false`, and the function returns `true`. Inactive projectiles are later removed from the `projectiles` vector.

5.5. Input Handling

- **GLUT Callbacks:** `glutKeyboardFunc`, `glutKeyboardUpFunc`, `glutSpecialFunc` (for arrow keys), and `glutSpecialUpFunc` are used to capture key presses and releases.
- **State-Dependent Logic:** The callback functions check the current `GameState` to determine the appropriate action (e.g., move car in `PLAYING`, navigate menu in `MENU`, exit on ESC).

5.6. State Management

A simple `enum class GameState` controls the overall flow. The `display` function renders different scenes based on the `currentState`, and the `update` function only progresses game logic when `currentState == GameState::PLAYING`. Input functions also modify behavior based on the state (e.g., 'P' toggles between `PLAYING` and `PAUSED`, Spacebar fires in `PLAYING` but restarts from `GAME_OVER`).

Code Highlights

Selected code snippets illustrating key implementation aspects.

6. Main Game Loop and State Update (main.cpp)

The `update` function drives game progression, calculating delta time and updating game objects based on the current state.

```
1 void update(int value) {
2     // Calculate delta time for frame-independent movement
3     float currentTime = glutGet(GLUT_ELAPSED_TIME) / 1000.0f;
4     deltaTime = currentTime - lastTime;
5     lastTime = currentTime;
6
7     // Only update game logic if in the PLAYING state
8     if (currentState == GameState::PLAYING) {
9         // Update the player car's position, projectiles, etc.
10        playerCar.update(deltaTime);
11
12        // Update the level: scroll road, move obstacles/
13        // enemies, spawn new ones
14        currentLevel.update(deltaTime, playerCar);
15
16        // Check for collisions between player/projectiles and
17        // obstacles/enemies
18        // Note: update handles collision consequences
19        // internally now
20        // currentLevel.checkCollisions(playerCar); // Might
21        // be redundant if update handles it
22
23        // Check for game over condition (player health <= 0)
24        if (playerCar.getHealth() <= 0) {
25            currentState = GameState::GAME_OVER;
26            // Optional: Save high score immediately upon game
27            // over
28            currentLevel.saveHighScore();
29        }
30    }
31
32    // Request a redraw for the next frame
33    glutPostRedisplay();
34
35    // Schedule the next update call
36    glutTimerFunc(16, update, 0); // Aim for ~60 FPS
37 }
```

Listing 1: Core Update Logic in `main.cpp`

7. Car Class (Car.h, Car.cpp)

Defines the vehicle properties and implements movement logic with boundary checks.

```
1  #pragma once
2  #include <GL/glew.h>
3  #include <vector>
4  #include "Texture.h" // Include Texture header
5
6  class Car {
7  public:
8      // Structure for projectiles fired by the car
9      struct Projectile {
10         float x, y;
11         float speed;
12         bool isActive;
13     };
14
15     Car();
16     ~Car();
17
18     void init(); // Initialize player car
19     void initAsEnemy(); // Initialize as an enemy car
20     void update(float deltaTime); // Update position,
    boundaries, projectiles
21     void render() const; // Render the car (player or enemy)
22
23     // Movement methods
24     void moveLeft();
25     void moveRight();
26     void moveUp();
27     void moveDown();
28     void stopHorizontal();
29     void stopVertical();
30
31     void fire(); // Fire a projectile
32
33     // Collision checking
34     bool checkCollision(float otherX, float otherY, float
    otherWidth, float otherHeight) const;
35     bool checkProjectileCollision(float targetX, float targetY
    , float targetWidth, float targetHeight) const; // Made
    const
36
37     // Game state methods
38     void takeDamage(float amount);
39     void addScore(int points); // Associated with the car
    instance
40     float getHealth() const;
41     int getScore() const; // Car keeps its own score? Maybe
    move to Level?
```

```

42     bool isActive() const;
43     void setActive(bool active);
44
45     // Getters for position and dimensions (using effective
46     // rendering sizes)
47     float getX() const;
48     float getY() const;
49     float getRenderWidth() const; // Use effective render/
50     // collision width
51     float getRenderHeight() const; // Use effective render/
52     // collision height
53
54     // Other getters/setters
55     void setX(float x);
56     void setY(float y);
57     float getSpeed() const; // Base speed for movement input
58     void setSpeed(float speed);
59     const std::vector<Projectile>& getProjectiles() const;
60
61     // Static members for shared enemy texture
62     static void ensureEnemyCarTextureLoaded();
63
64 private:
65     float x, y; // Center position
66     // const float width = 0.1f; // Original member - seems
67     // unused for rendering/collision
68     // const float height = 0.1f; // Original member - seems
69     // unused for rendering/collision
70     float speed; // Base movement speed factor
71     float health;
72     int score; // Score associated with this car (
73     // usually player)
74     bool active; // Is the car currently active in the
75     // game?
76     bool isEnemy = false; // Flag to distinguish player/enemy
77
78     // Movement state
79     float horizontalSpeed = 0.0f; // Current velocity
80     // component X
81     float verticalSpeed = 0.0f; // Current velocity
82     // component Y
83
84     // Projectile firing state
85     const float fireCooldown = 0.5f; // Time between shots
86     float fireTimer = 0.0f; // Time remaining until
87     // next shot
88     std::vector<Projectile> projectiles; // Projectiles fired
89     // by this car
90     static constexpr float PROJECTILE_SPEED = 10.0f;
91
92     // Textures

```

```
82     Texture carTexture; // Player texture
83     static Texture enemyCarTexture; // Shared texture for all
84     enemies
85     static bool enemyCarTextureLoaded; // Flag for static
86     texture loading
87
88     // Effective dimensions used for rendering and collision
89     // These match the hardcoded values in render() and
90     checkCollision()
91     static constexpr float RENDER_WIDTH_PLAYER = 0.2f;
92     static constexpr float RENDER_HEIGHT_PLAYER = 0.4f;
93     static constexpr float RENDER_WIDTH_ENEMY =
94     RENDER_WIDTH_PLAYER * 0.8f;
95     static constexpr float RENDER_HEIGHT_ENEMY =
96     RENDER_HEIGHT_PLAYER * 0.8f;
97 };
```

Listing 2: Car Class Definition Snippet (Car.h)

```
1 void Car::update(float deltaTime) {
2     // Calculate potential new position based on current
3     velocity
4     float newX = x + horizontalSpeed * deltaTime;
5     float newY = y + verticalSpeed * deltaTime;
6
7     // --- Boundary Calculation (Crucial for Correctness) ---
8     // These values MUST align with Level::drawLaneLines and
9     obstacle spawning
10    // Road width is 4.5f in Level class
11    const float ROAD_WIDTH = 4.5f;
12    const float EMERGENCY_MARGIN = 0.3f; // From Level::
13    drawLaneLines
14    float yellowLinePos = ROAD_WIDTH - EMERGENCY_MARGIN; //
15    Adjusted logic based on Level
16    // Correct effective car dimensions for boundary check
17    float effectiveCarWidth = isEnemy ? RENDER_WIDTH_ENEMY :
18    RENDER_WIDTH_PLAYER;
19    float effectiveCarHeight = isEnemy ? RENDER_HEIGHT_ENEMY :
20    RENDER_HEIGHT_PLAYER;
21    const float SAFETY_MARGIN = 0.01f; // Smaller safety
22    margin if needed
23
24    // Calculate bounds (using center position)
25    float leftBound = -yellowLinePos + effectiveCarWidth / 2.0
26    f + SAFETY_MARGIN;
27    float rightBound = yellowLinePos - effectiveCarWidth / 2.0
28    f - SAFETY_MARGIN;
29
30    // Vertical bounds (ensure car stays roughly in the lower
31    part of the screen)
32    // These bounds seem somewhat arbitrary in the original
33    code; adjust as needed.
```

```
23     float bottomBound = -1.8f + effectiveCarHeight / 2.0f;
24     float topBound = -0.5f - effectiveCarHeight / 2.0f; //
    Limit upward movement
25
26     // --- Apply Movement with Boundary Clamping ---
27     // Horizontal Movement
28     if (newX < leftBound) {
29         x = leftBound;
30         horizontalSpeed = 0.0f; // Stop movement when hitting
    boundary
31     } else if (newX > rightBound) {
32         x = rightBound;
33         horizontalSpeed = 0.0f;
34     } else {
35         x = newX; // Apply movement normally
36     }
37
38     // Vertical Movement (Only if NOT an enemy, enemies move
    via Level::update)
39     if (!isEnemy) {
40         if (newY < bottomBound) {
41             y = bottomBound;
42             verticalSpeed = 0.0f;
43         } else if (newY > topBound) {
44             y = topBound;
45             verticalSpeed = 0.0f;
46         } else {
47             y = newY;
48         }
49     }
50     // Note: Enemy Y position is updated in Level::update
    based on scroll speed
51
52     // --- Update Firing Cooldown ---
53     if (fireTimer > 0.0f) {
54         fireTimer -= deltaTime;
55     }
56
57     // --- Update Projectiles ---
58     for (auto& projectile : projectiles) {
59         if (projectile.isActive) {
60             projectile.y += PROJECTILE_SPEED * deltaTime;
61             // Check if projectile goes off-screen (adjust top
    boundary as needed)
62             if (projectile.y > 2.0f) { // Assuming screen top
    is around Y=1.0 to 2.0
63                 projectile.isActive = false;
64             }
65         }
66     }
67
```

```

68     // --- Remove Inactive Projectiles ---
69     projectiles.erase(
70         std::remove_if(projectiles.begin(), projectiles.end(),
71             [](const Projectile& p) { return !p.isActive; }),
72         projectiles.end()
73     );
74 }

```

Listing 3: Car Movement Update Snippet (Car.cpp)

8. Level Class (Level.h, Level.cpp)

Manages the game world, including road rendering, obstacle/enemy spawning, and scoring.

```

1  #pragma once
2  #include <GL/glew.h>
3  #include <vector>
4  #include <string>
5  #include "Car.h"           // Include Car header
6  #include "Texture.h"      // Include Texture header
7
8  class Level {
9  public:
10     // Enum to define different types of obstacles
11     enum class ObstacleType {
12         BARRIER, CONE, ROCK, TREE // Removed ENEMY_CAR as they
13         // are separate Car objects
14     };
15
16     // Structure to hold obstacle data
17     struct Obstacle {
18         float x, y;           // Position
19         float width, height;  // Dimensions
20         ObstacleType type;    // Type for rendering/collision
21         // behavior
22         bool isActive;        // Whether the obstacle is
23         // currently in play
24         // Texture* texture; // Pointer to the relevant
25         // texture (set during spawn)
26     };
27
28     Level();
29     ~Level();
30
31     void init(); // Load textures, initialize state
32     void update(float deltaTime, Car& playerCar); // Update
33     // scrolling, objects, spawns, score
34     void render(); // Render the road, obstacles, enemies
35     void reset(); // Reset level to initial state for a new

```

```
game
31
32 // Getters for game state info
33 int getScore() const;
34 int getLevel() const;
35 int getHighScore() const;
36 void saveHighScore(); // Exposed for saving on game over
    potentially
37
38 // Collision checking orchestrated by Level
39 void checkCollisions(Car& playerCar);
40
41 private:
42 // Road rendering methods
43 void createRoadTexture(); // Procedurally generate road
    texture
44 void drawRoad();
45 void drawLaneLines();
46
47 // Spawning methods
48 void spawnObstacle();
49 void spawnEnemyCar();
50
51 // Rendering helper
52 void drawObstacles();
53
54 // Internal state updates
55 void updateLevelLogic(); // Update the level number based
    on score
56 void updateScore(float deltaTime); // Update score based
    on time and speed
57 void loadHighScore(); // Load high score from file
58
59 // --- Member Variables ---
60 GLuint roadTextureId = 0; // OpenGL texture ID for the
    road
61 std::vector<Obstacle> obstacles; // Active obstacles
62 std::vector<Car> enemyCars; // Active enemy cars
63
64 // Road properties
65 float roadWidth;
66 float roadLength; // Used for spawning Y coordinate and
    despawn check
67 float scrollSpeed; // Current speed of the road/obstacles
68 float roadOffset = 0.0f; // Texture coordinate offset for
    scrolling effect
69
70 // Game state
71 int score;
72 int currentLevelNum; // Renamed from 'level' to avoid
    conflict
```

```

73     int highScore;
74
75     // Timers
76     float levelTimer = 0.0f; // Total time elapsed in the
level (unused?)
77     float obstacleSpawnTimer = 0.0f; // Time since last
obstacle spawn
78     float enemySpawnTimer = 0.0f; // Time since last enemy
spawn
79     float scoreUpdateTimer = 0.0f; // Timer for periodic score
increase
80
81     // Textures for different obstacle types
82     Texture barrierTexture;
83     Texture coneTexture;
84     Texture rockTexture;
85     Texture treeTexture;
86     // Note: Enemy car texture is static in Car class
87
88     // Constants (Consider moving to a separate Constants.h)
89     static constexpr float INITIAL_SCROLL_SPEED = 0.5f; //
Starting speed
90     static constexpr float MAX_SCROLL_SPEED = 2.0f;
91     static constexpr float SPEED_INCREASE_PER_SEC = 0.04f;
92     static constexpr int INITIAL_SCORE = 0; // Start score at
0
93     static constexpr int LEVEL_UP_SCORE_BASE = 100; // Score
needed for level 2
94     static constexpr int LEVEL_UP_MULTIPLIER = 2; // Score
threshold doubles each level
95     static constexpr float SCORE_PER_SECOND = 1.0f;
96     static constexpr float SPEED_SCORE_MULTIPLIER = 0.5f;
97     static constexpr float INITIAL_OBSTACLE_SPAWN_INTERVAL =
1.0f;
98     static constexpr float MIN_OBSTACLE_SPAWN_INTERVAL = 0.1f;
99     static constexpr float OBSTACLE_SPAWN_SPEED_FACTOR = 0.5f;
// How speed affects spawn rate
100    static constexpr float ENEMY_SPAWN_INTERVAL_BASE = 4.0f;
// Base time between enemy spawns
101    static constexpr int ROAD_TEXTURE_SIZE = 256;
102    static constexpr int OBSTACLE_DESTROY_POINTS = 10; //
Points for destroying obstacle
103    static constexpr int ENEMY_DESTROY_POINTS = 50; // Points
for destroying enemy
104    const char* HIGH_SCORE_FILE = "highscore.txt";
105 };

```

Listing 4: Level Class Definition Snippet (Level.h)

```

1 void Level::spawnObstacle() {
2     Obstacle obstacle;
3     obstacle.isActive = true;

```

```
4     obstacle.y = roadLength; // Spawn at the top edge
5
6     // --- Calculate Valid Spawn Range (Consistent with Car
7     // Must match values in Car::update and Level::
8     drawLaneLines
9     const float EMERGENCY_MARGIN = 0.3f;
10    float yellowLinePos = roadWidth - EMERGENCY_MARGIN;
11    const float SAFETY_MARGIN = 0.1f; // Margin from the
12    yellow line
13
14    // Determine max obstacle width to calculate bounds
15    correctly
16    // (Using a fixed estimate here, could be dynamic)
17    const float MAX_OBSTACLE_WIDTH_ESTIMATE = 0.3f;
18    float leftBound = -yellowLinePos +
19    MAX_OBSTACLE_WIDTH_ESTIMATE / 2.0f + SAFETY_MARGIN;
20    float rightBound = yellowLinePos -
21    MAX_OBSTACLE_WIDTH_ESTIMATE / 2.0f - SAFETY_MARGIN;
22
23    // Ensure bounds are valid
24    if (leftBound >= rightBound) {
25        // Fallback or error logging if road is too narrow for
26        safe spawning
27        obstacle.x = 0.0f; // Spawn in center if bounds
28        invalid
29    } else {
30        // Generate random X within the calculated bounds
31        obstacle.x = leftBound + static_cast<float>(rand()) /
32        RAND_MAX * (rightBound - leftBound);
33    }
34
35    // Randomly select obstacle type and set its specific
36    dimensions
37    int typeIndex = rand() % 4; // Assuming 4 types: BARRIER,
38    CONE, ROCK, TREE
39    switch (typeIndex) {
40        case 0:
41            obstacle.type = ObstacleType::BARRIER;
42            obstacle.width = 0.3f; obstacle.height = 0.3f;
43            // obstacle.texture = &barrierTexture; // Assign
44            texture pointer
45            break;
46        case 1:
47            obstacle.type = ObstacleType::CONE;
48            obstacle.width = 0.2f; obstacle.height = 0.2f;
49            // obstacle.texture = &coneTexture;
50            break;
51        case 2:
52            obstacle.type = ObstacleType::ROCK;
53            obstacle.width = 0.25f; obstacle.height = 0.25f;
```



```
43         // obstacle.texture = &rockTexture;
44         break;
45     case 3:
46         obstacle.type = ObstacleType::TREE;
47         obstacle.width = 0.25f; obstacle.height = 0.25f;
48         // obstacle.texture = &treeTexture;
49         break;
50     }
51     // Ensure spawned obstacle width doesn't violate
52     // (Refinement: adjust spawn bounds based on chosen
53     // obstacle's actual width)
54     float halfWidth = obstacle.width / 2.0f;
55     if (obstacle.x - halfWidth < leftBound + SAFETY_MARGIN)
56         obstacle.x = leftBound + halfWidth + SAFETY_MARGIN;
57     if (obstacle.x + halfWidth > rightBound - SAFETY_MARGIN)
58         obstacle.x = rightBound - halfWidth - SAFETY_MARGIN;
59     obstacles.push_back(obstacle);
60 }
```

Listing 5: Level Obstacle Spawning (Level.cpp)

9. Texture Handling (Texture.h, Texture.cpp)

The `Texture` class simplifies loading images and managing OpenGL texture objects.

```
1 #include "../include/Texture.h"
2 #define STB_IMAGE_IMPLEMENTATION // Define only once in your
3 // project
4 #include "../include/stb_image.h" // Include stb_image header
5 #include <iostream>
6
7 Texture::Texture() : textureID(0), width(0), height(0),
8 channels(0) {}
9
10 Texture::~Texture() {
11     // Release OpenGL texture resource if it was created
12     if (textureID != 0) {
13         glDeleteTextures(1, &textureID);
14     }
15 }
16
17 bool Texture::loadFromFile(const std::string& filename) {
18     // Ensure vertical flip for OpenGL coordinates (bottom-
19     // left origin)
20     stbi_set_flip_vertically_on_load(true);
21
22     // Load image data using stb_image
```

```
20     unsigned char* data = stbi_load(filename.c_str(), &width,
21                                     &height, &channels, 0);
22
23     if (!data) {
24         std::cerr << "ERROR::TEXTURE: Failed to load texture
25         file: " << filename << std::endl;
26         return false;
27     }
28
29     std::cout << "Texture Info: Loaded '" << filename << "' ("
30     << width << "x" << height << "), Channels: " << channels
31     << std::endl;
32
33     // Determine OpenGL texture format based on number of
34     channels
35     GLenum format;
36     GLenum internalFormat; // Format OpenGL uses internally
37     if (channels == 4) {
38         format = GL_RGBA;
39         internalFormat = GL_RGBA; // Or GL_RGBA8 for specific
40         sizing
41         std::cout << " -> Using RGBA format." << std::endl;
42     } else if (channels == 3) {
43         format = GL_RGB;
44         internalFormat = GL_RGB; // Or GL_RGB8
45         std::cout << " -> Using RGB format." << std::endl;
46     } else if (channels == 1) {
47         format = GL_RED; // Use RED for single channel (
48         grayscale)
49         internalFormat = GL_RED;
50         std::cout << " -> Using RED format (Grayscale)." <<
51         std::endl;
52     } else {
53         std::cerr << "ERROR::TEXTURE: Unsupported number of
54         channels (" << channels << ") in file: " << filename << std
55         ::endl;
56         stbi_image_free(data); // Free stb_image memory
57         return false;
58     }
59
60     // Generate OpenGL texture object
61     glGenTextures(1, &textureID);
62     glBindTexture(GL_TEXTURE_2D, textureID);
63
64     // Set texture wrapping parameters (GL_REPEAT for tiling
65     road)
66     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
67     GL_REPEAT);
68     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
69     GL_REPEAT);
```

```
58     // Set texture filtering parameters (GL_LINEAR for smooth
59     scaling)
60     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
61     GL_LINEAR_MIPMAP_LINEAR); // Use mipmaps for minification
62     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
63     GL_LINEAR);
64
65     // Upload texture data to GPU
66     glTexImage2D(GL_TEXTURE_2D, 0, internalFormat, width,
67     height, 0, format, GL_UNSIGNED_BYTE, data);
68
69     // Generate mipmaps for better quality at smaller sizes
70     glGenerateMipmap(GL_TEXTURE_2D);
71
72     // Free the image data loaded by stb_image
73     stbi_image_free(data);
74
75     // Unbind texture (optional, good practice)
76     glBindTexture(GL_TEXTURE_2D, 0);
77
78     return true;
79 }
80
81 void Texture::bind() const {
82     if (textureID != 0) {
83         glBindTexture(GL_TEXTURE_2D, textureID);
84     }
85 }
86
87 void Texture::unbind() const {
88     glBindTexture(GL_TEXTURE_2D, 0);
89 }
```

Listing 6: Texture Loading Implementation (Texture.cpp)

Key Gameplay Mechanics

Detailed exploration of the core gameplay systems.

10. Dynamic Road System

The illusion of continuous forward motion is central to the game. This is achieved by:

- **Scrolling Texture:** A seamless, tileable road texture is mapped onto a long quad representing the road surface. The vertical texture coordinate (*v*) is offset each frame based on the *scrollSpeed* and *deltaTime*. The *GL_REPEAT* texture wrapping parameter ensures the texture tiles correctly as the offset increases.
- **Lane Lines:** Dashed white center lines and solid yellow edge lines are drawn separately using *glBegin(GL_LINES)*. Their vertical positions are also offset using the same logic as the road texture's coordinates, ensuring they scroll in perfect synchronization with the road surface, maintaining visual coherence. The line widths and colors are set using *glLineWidth* and *glColor3f*. The positions of the yellow lines are critical as they define the logical boundaries for player movement and object spawning.

```
1 void Level::drawRoad() {
2     glEnable(GL_TEXTURE_2D);
3     glBindTexture(GL_TEXTURE_2D, roadTextureId); // Bind the
4     procedural road texture
5     glColor3f(1.0f, 1.0f, 1.0f); // Use white color to show
6     texture faithfully
7
8     // Define texture coordinates based on roadOffset for
9     scrolling
10    float texTop = roadOffset * 5.0f; // Multiply offset for
11    faster perceived scroll
12    float texBottom = 0.0f; // Bottom edge fixed
13
14    glBegin(GL_QUADS);
15    // Bottom-left
16    glTexCoord2f(0.0f, texBottom); glVertex3f(-roadWidth, -
17    roadLength, -0.1f);
18    // Bottom-right
19    glTexCoord2f(1.0f, texBottom); glVertex3f( roadWidth, -
20    roadLength, -0.1f);
21    // Top-right (Scrolling coordinate)
22    glTexCoord2f(1.0f, texTop); glVertex3f( roadWidth,
23    roadLength, -0.1f);
24    // Top-left (Scrolling coordinate)
25    glTexCoord2f(0.0f, texTop); glVertex3f(-roadWidth,
26    roadLength, -0.1f);
27 }
```

```
19     glEnd();
20
21     glBindTexture(GL_TEXTURE_2D, 0); // Unbind texture
22     glDisable(GL_TEXTURE_2D);
23
24     // Draw lane lines on top of the road texture
25     drawLaneLines();
26 }
27
28 void Level::drawLaneLines() {
29     // Define line positions and appearance
30     const float EMERGENCY_MARGIN = 0.3f; // Margin from
    physical road edge
31     float yellowLinePos = roadWidth - EMERGENCY_MARGIN; //
    Position from center
32     const float DASH_LENGTH = 0.5f;
33     const float GAP_LENGTH = 0.5f;
34     const float LINE_CYCLE = DASH_LENGTH + GAP_LENGTH;
35
36     // Calculate offset for synchronized scrolling with road
    texture
37     // Use modulo to keep offset within one cycle length
38     float lineOffset = fmod(roadOffset * 5.0f * LINE_CYCLE,
    LINE_CYCLE);
39
40     // Draw white dashed center line
41     glColor3f(1.0f, 1.0f, 1.0f); // White
42     glLineWidth(6.0f); // Thickness
43     glBegin(GL_LINES);
44     for (float y = -roadLength - lineOffset; y < roadLength; y
    += LINE_CYCLE) {
45         float yStart = y;
46         float yEnd = y + DASH_LENGTH;
47         // Clip lines to be within the visible road segment [-
    roadLength, roadLength]
48         glVertex3f(0.0f, std::max(-roadLength, yStart), -0.05f
    );
49         glVertex3f(0.0f, std::min( roadLength, yEnd), -0.05f);
50     }
51     glEnd();
52
53     // Draw solid yellow boundary lines
54     glColor3f(1.0f, 1.0f, 0.0f); // Yellow
55     glLineWidth(8.0f); // Thickness
56     glBegin(GL_LINES);
57     // Left boundary line
58     glVertex3f(-yellowLinePos, -roadLength, 0.0f);
59     glVertex3f(-yellowLinePos,  roadLength, 0.0f);
60     // Right boundary line
61     glVertex3f( yellowLinePos, -roadLength, 0.0f);
62     glVertex3f( yellowLinePos,  roadLength, 0.0f);
```

```
63     glEnd();
64
65     glLineWidth(1.0f); // Reset line width
66 }
```

Listing 7: Road and Lane Line Rendering Snippet (Level.cpp)

11. Obstacle and Enemy System

Dynamic challenges are introduced through randomly spawned obstacles and faster enemy cars.

- **Spawning Logic:** The `Level::update` function uses timers (`obstacleSpawnTimer`, `enemySpawnTimer`) and intervals that decrease as the `scrollSpeed` increases. When a timer exceeds its interval, the corresponding spawn function (`spawnObstacle` or `spawnEnemyCar`) is called, and the timer resets.
- **Placement:** Spawn functions calculate a valid horizontal position (`x`) within the yellow lane boundaries (using the same boundary logic as the player car) and set the initial vertical position (`y`) at the top of the visible area (`roadLength`).
- **Variety:** `spawnObstacle` randomly selects an `ObstacleType` and assigns appropriate dimensions and texture. `spawnEnemyCar` creates a `Car` object initialized using `initAsEnemy`.
- **Movement:** Obstacles move downwards with the `scrollSpeed`. Enemy cars are updated separately in `Level::update` to move downwards at `scrollSpeed * 2.0f` (or another multiplier).
- **Despawning:** Objects are implicitly removed when they move far below the screen (`y < -roadLength`). Active obstacles/enemies involved in collisions are flagged as inactive (`setActive(false)` or `obstacle.isActive = false`). The `checkCollisions` function or update loops should handle removal of inactive objects from their respective vectors (e.g., using the erase-remove idiom, as seen in `Car::update` for projectiles).

```
1 void Level::spawnEnemyCar() {
2     Car enemy; // Create a new Car object
3
4     // Calculate spawn boundaries (consistent with player car
5     // and obstacles)
6     const float EMERGENCY_MARGIN = 0.3f;
7     float yellowLinePos = roadWidth - EMERGENCY_MARGIN;
8     const float SAFETY_MARGIN = 0.1f;
9     // Use enemy car's specific render width for bounds
10    // calculation
11    float enemyRenderWidth = Car::RENDER_WIDTH_ENEMY;
12    float leftBound = -yellowLinePos + enemyRenderWidth / 2.0f
13    + SAFETY_MARGIN;
```

```

11     float rightBound = yellowLinePos - enemyRenderWidth / 2.0f
    - SAFETY_MARGIN;
12
13     // Set initial position (random X within bounds, Y at the
    top)
14     if (leftBound >= rightBound) {
15         enemy.setX(0.0f); // Fallback if bounds invalid
16     } else {
17         enemy.setX(leftBound + static_cast<float>(rand()) /
RAND_MAX * (rightBound - leftBound));
18     }
19     enemy.setY(roadLength); // Spawn at the top edge
20
21     // Initialize specific enemy properties
22     enemy.setActive(true);
23     enemy.initAsEnemy(); // Crucial: sets 'isEnemy' flag and
loads correct texture (statically)
24
25     // Note: Enemy speed (downward movement) is handled in
Level::update
26     // based on the current scrollSpeed. We don't set
verticalSpeed here.
27
28     enemyCars.push_back(enemy); // Add the new enemy car to
the level's list
29 }

```

Listing 8: Enemy Car Spawning (Level.cpp)

12. Scoring, Progression, and High Score

The game motivates players through scoring and increasing challenge.

- **Time/Speed Score:** `Level::updateScore` increments the `score` periodically (e.g., every second). The amount added includes a base value (`SCORE_PER_SECOND`) plus a bonus proportional to the current `scrollSpeed` (`SPEED_SCORE_MULTIPLIER`).
- **Action Score:** Points are added to the `score` immediately when the player's projectiles destroy an obstacle (`OBSTACLE_DESTROY_POINTS`) or an enemy car (`ENEMY_DESTROY_POINTS`) within the `Level::checkCollisions` function.
- **Level Progression:** `Level::updateLevelLogic` checks if the `score` has passed thresholds calculated using `LEVEL_UP_SCORE_BASE` and `LEVEL_UP_MULTIPLIER`. The `currentLevelNum` is updated, which is displayed on the HUD. While the level number changes, the primary difficulty increase comes from the continuously rising `scrollSpeed`.
- **Speed Increase:** The `scrollSpeed` itself increases gradually over time (`SPEED_INCREASE_PER_SEC * deltaTime`) up to `MAX_SCROLL_SPEED`, directly impacting game difficulty.

- **High Score:** `Level::loadHighScore` reads the best score from `highscore.txt` during initialization. `Level::saveHighScore` is called (e.g., in `update` when game over occurs, or in `reset`) if the current `score` exceeds the `highScore`, overwriting the file.

```
1 void Level::updateScore(float deltaTime) {
2     scoreUpdateTimer += deltaTime;
3     if (scoreUpdateTimer >= 1.0f) { // Update score roughly
4         // Base score for survival time
5         int baseScore = static_cast<int>(SCORE_PER_SECOND);
6         // Bonus score based on current speed
7         int speedBonus = static_cast<int>(scrollSpeed *
8             SPEED_SCORE_MULTIPLIER);
9
10        score += (baseScore + speedBonus);
11
12        scoreUpdateTimer -= 1.0f; // Reset timer preserving
13        // any overshoot
14
15        // Update high score if current score is higher
16        if (score > highScore) {
17            highScore = score;
18            // Consider saving high score less frequently, e.g
19            // .. only on game over
20            // saveHighScore(); // Saving every second might
21            // be inefficient
22        }
23    }
24 }
25
26 void Level::loadHighScore() {
27     std::ifstream file(HIGH_SCORE_FILE);
28     if (file.is_open()) {
29         file >> highScore;
30         file.close();
31         std::cout << "Loaded high score: " << highScore << std
32         ::endl;
33     } else {
34         highScore = 0; // Default high score if file doesn't
35         // exist
36         std::cout << "High score file not found. Setting high
37         score to 0." << std::endl;
38     }
39 }
40
41 void Level::saveHighScore() {
42     // Only save if the current score is indeed the highest
43     // recorded
44     // This prevents overwriting with a lower score if called
45     // erroneously
```



```
37     // We might need to re-read the file here for absolute
    certainty in concurrent scenarios (unlikely here)
38     if (score > highScore) {
39         highScore = score; // Ensure internal highScore is up
    -to-date
40     }
41     // Now save the current highScore value
42     std::ofstream file(HIGH_SCORE_FILE);
43     if (file.is_open()) {
44         file << highScore;
45         file.close();
46         std::cout << "Saved new high score: " << highScore <<
    std::endl;
47     } else {
48         std::cerr << "ERROR: Failed to open high score file
    for writing: " << HIGH_SCORE_FILE << std::endl;
49     }
50 }
51
52 void Level::updateLevelLogic() {
53     // Calculate score threshold for the *next* level
54     int nextLevelThreshold = LEVEL_UP_SCORE_BASE * static_cast
    <int>(pow(LEVEL_UP_MULTIPLIER, currentLevelNum));
55
56     if (score >= nextLevelThreshold) {
57         currentLevelNum++;
58         std::cout << "Level Up! Reached Level: " <<
    currentLevelNum << std::endl;
59         // Optional: Trigger special effects or slightly
    change game parameters on level up
60     }
61 }
```

Listing 9: Scoring Update and High Score Logic (Level.cpp)

Challenges and Solutions

Discussion of difficulties encountered during development and the strategies used to overcome them.

13. Performance Optimization

Maintaining a smooth frame rate (targeting 60 FPS) became challenging as the number of objects (obstacles, enemies, projectiles) on screen increased, especially with the rising game speed.

- **Challenge:** Increased draw calls and physics updates per frame leading to performance degradation. Texture loading and binding overhead.
- **Solution Approaches:**
 - **Efficient Texture Management:** Using the `Texture` class to load textures once during initialization (`Level::init`) and simply binding existing texture IDs during rendering (`Texture::bind`). Using static members for shared textures (like `enemyCarTexture`) ensures they are loaded only once across all instances.
 - **Optimized Object Removal:** Implementing efficient removal of inactive objects (projectiles, obstacles, enemies) from their `std::vector` containers using the erase-remove idiom (`vector.erase(std::remove_if(...), vector.end())`) is generally better than manual iteration and erasing.
 - **Simplified Rendering:** Sticking to the fixed-function pipeline and simple geometry (quads) avoids the overhead of complex shader compilation and VBO management, which can be simpler for this type of game, though less flexible than modern OpenGL.
 - *(Potential Improvement):* Implementing object pooling for obstacles and projectiles instead of dynamic allocation/deallocation (`push_back/erase`) could further reduce overhead. Frustum culling (checking if an object is within the viewable area before drawing) could also help if the game world expanded significantly, though it's less critical in this top-down scrolling view.

14. Collision Detection Accuracy and Consistency

Ensuring collisions felt fair and consistent was vital for gameplay.

- **Challenge:** AABB collisions can feel inaccurate for non-rectangular shapes. Ensuring collision boundaries perfectly matched visual boundaries and movement limits. Minor discrepancies could lead to frustrating "ghost" collisions or passing through objects.
- **Solution Approaches:**

- **Consistent Boundary Definitions:** The most critical step was ensuring that the boundaries used for player movement clamping (`Car::update`), obstacle/enemy spawning (`Level::spawnObstacle`, `Level::spawnEnemyCar`), and visual lane line rendering (`Level::drawLaneLines`) were all derived from the *same* base values (`roadWidth`, `EMERGENCY_MARGIN`) and calculations. Any inconsistency here breaks the system.
- **Accurate Sizing:** Using the correct effective dimensions (the visual/collision size, e.g., `RENDER_WIDTH_PLAYER`, not necessarily the original `Car::width` member) in both the `Car::checkCollision` calculations and the boundary checks in `Car::update` was essential.
- **Safety Margins:** Small safety margins (`SAFETY_MARGIN`) were added in boundary calculations to provide a slight buffer, preventing objects or the player from appearing to precisely touch or slightly cross visual lines before stopping.
- *(Potential Improvement):* For more complex shapes, using multiple bounding boxes per object or switching to circle-based or pixel-perfect collision detection (much more complex) could improve accuracy, but AABB was deemed sufficient for this arcade style.

15. Visual Cohesion (Scrolling Synchronization)

Making the road texture, lane lines, obstacles, and enemies scroll together smoothly was necessary for immersion.

- **Challenge:** Any difference in the calculation of movement or texture coordinate updates between these elements would cause visual jittering or elements drifting apart.
- **Solution Approaches:**
 - **Centralized Speed Control:** The `Level` class manages the single `scrollSpeed` variable.
 - **Delta Time Usage:** All movement calculations (player input, object scrolling, texture coordinate offsets) consistently use the calculated `deltaTime` to ensure frame rate independence and smooth motion.
 - **Shared Offset Logic:** The texture coordinate offset (`roadOffset`) used for the road texture (`Level::drawRoad`) and the calculation for the lane line drawing positions (`Level::drawLaneLines`) are derived from the same `scrollSpeed * deltaTime` logic (potentially with multipliers like `* 5.0f` applied consistently) to guarantee synchronization. Obstacles and enemies also have their `y` positions updated based directly on `scrollSpeed` and `deltaTime`.

Conclusion and Future Work

16. Project Summary

NeedForSpeedInfinity successfully meets the objectives set out for the Computer Graphics 2nd inter semester activity project. It delivers a functional and engaging arcade racing experience built upon fundamental OpenGL principles. Key graphics techniques like texture mapping, orthographic projection, animation through scrolling, AABB collision detection, and game state management were effectively implemented using C++, GLUT, and GLEW. The resulting game features a dynamic environment with progressive difficulty, responsive controls, clear visual feedback through the HUD, and persistent high scores, offering a solid foundation and a replayable core loop.

17. Learning Outcomes

The development process provided significant practical learning in several areas:

- ★ **OpenGL Fundamentals:** Hands-on application of the fixed-function pipeline, including vertex specification, color application, texture mapping (`glTexCoord`, `glBindTexture`), transformations (implicit through projection), and state management (`glEnable`, `glDisable`).
- ★ **Game Loop Architecture:** Implementing a robust game loop using timer callbacks, managing delta time for frame-rate independence, and structuring update and render phases.
- ★ **Object-Oriented Game Design:** Utilizing classes (`Car`, `Level`, `Texture`) to encapsulate data and behavior, promoting modularity and code reuse. Understanding the use of static members for shared resources.
- ★ **Collision Detection:** Implementing and debugging AABB collision detection and response.
- ★ **Input Handling:** Processing keyboard events for real-time control using GLUT callbacks.
- ★ **State Management:** Designing and implementing finite state machines for controlling game flow (Menu, Play, Pause, Game Over).
- ★ **Debugging Graphics Applications:** Identifying and resolving issues related to rendering artifacts, incorrect transformations, collision inaccuracies, and performance bottlenecks.
- ★ **External Libraries Integration:** Incorporating third-party libraries like GLEW, GLUT, and stb_image into the project build process.

18. Future Enhancements

While the current version provides a complete core experience, several avenues exist for future expansion:

- **Audio Integration:** Adding sound effects (engine sounds, collisions, firing, menu clicks) and background music using a library like OpenAL or SDL_mixer would significantly enhance immersion.
- **Visual Polish:** * Implementing particle effects for collisions, explosions, or tire smoke. * Adding simple lighting effects or shadows, even in the fixed-function pipeline, or transitioning to modern OpenGL with shaders for advanced effects. * Using sprite sheets for animated effects or car states.
- **Gameplay Variety:** * Introducing power-ups (speed boost, shield, temporary weapon upgrades). * Adding different types of terrain or road segments (tunnels, bridges) with varying visual themes. * Implementing more sophisticated enemy AI patterns (e.g., lane changing, dodging).
- **Modernization:** Migrating the rendering pipeline to modern OpenGL (using shaders, VBOs, VAOs) for improved performance and greater graphical flexibility.
- **User Interface:** Enhancing the menu system with more options (settings, controls display) and improving the visual design of the HUD and menus.
- **Code Refinement:** Refactoring hardcoded values into constants or configuration files. Improving error handling and logging. Addressing potential issues like the `const_cast` usage.

References

Resources consulted during the project development:

- [1] *OpenGL Programming Guide (The Red Book)* - Official guide for learning OpenGL (various editions cover different versions).
- [2] *LearnOpenGL* website by Joey de Vries - <https://learnopengl.com/> (Excellent resource for modern OpenGL, concepts are often transferable).
- [3] *FreeGLUT Documentation* - <http://freeglut.sourceforge.net/docs/api.php> (API reference for GLUT functions).
- [4] *GLEW (The OpenGL Extension Wrangler Library)* - <http://glew.sourceforge.net/> (For accessing OpenGL extensions).
- [5] *stb_image.h* by Sean Barrett - https://github.com/nothings/stb/blob/master/stb_image.h (Single-file library for image loading).
- [6] *Game Programming Patterns* by Robert Nystrom - <https://gameprogrammingpatterns.com/> (Useful design patterns for game development, e.g., Game Loop, State, Object Pooling).
- [7] *Freepik*, <https://www.freepik.com/> - Various PNG assets used in the project.
- [8] *Sketchup Texture Club*, <https://www.sketchuptextureclub.com/> - Resources for 3D textures.
- [9] *FreeSVG*, <https://freesvg.org/top-view-car-vector> - Car vector image used in the project.
- [10] *PNGTree*, <https://pngtree.com/> - Resources for PNG assets.
- [11] *Space Shooter C++ Code* by Sarthak Pokharel - <https://github.com/sarthak-pokharel/space-shooter-sdl> - Inspiration for shooting and projectile mechanics in the game.
- [12] *C++ OpenGL Setup for VSCode in 2 Minutes* - <https://youtu.be/Y4F0tI7WlDs> - A tutorial on setting up OpenGL in VSCode for C++ development.
- [13] *FreeGLUT Development Downloads* - <https://www.transmissionzero.co.uk/software/freeglut-devel/> - Development versions of FreeGLUT.
- [14] *GLEW Downloads* - <https://sourceforge.net/projects/glew/files/glew/2.0/> - GLEW library for managing OpenGL extensions.
- [15] *Various online forums and communities like Stack Overflow, OpenGL forums, GameDev.net for specific troubleshooting.*
- [16] *AI Tools and Agents Used: Cursor, Claude, ChatGPT, Gemini, GitHub Copilot.*

Appendix

19. Project Directory Structure

The project source code and assets are organized as follows:

```
NeedForSpeedInfinity/  
|-- NeedForSpeedInfinity.sln          # Visual Studio Solution File  
|-- NeedForSpeedInfinity.vcxproj      # Visual Studio Project File  
|-- main.cpp                         # Main application entry point, game loop, state mgt  
|-- highscore.txt                   # Stores the persistent high score  
|-- README.md                       # Project overview and instructions  
  
|-- include/                         # Header files (.h)  
|   |-- Car.h                       # Car class definition  
|   |-- Level.h                     # Level class definition  
|   |-- Texture.h                   # Texture class definition  
|   |-- resource.h                  # Windows resource header (if using .rc file)  
|   '-- stb_image.h                 # STB Image library header  
  
|-- src/                             # Source files (.cpp)  
|   |-- Car.cpp                     # Car class implementation  
|   |-- Level.cpp                   # Level class implementation  
|   '-- Texture.cpp                 # Texture class implementation  
  
|-- textures/                        # Image assets (.png, .jpg, etc.)  
|   |-- car.png  
|   |-- enemycar.png  
|   |-- barricade.png  
|   |-- cone.png  
|   |-- rock.png  
|   |-- tree.png  
|   '-- [Other textures...]  
  
|-- lib/                             # Pre-compiled libraries (optional, depends on setup)  
|   |-- freeglut.lib  
|   '-- glew32.lib  
  
|-- bin/ or x64/                     # Compiled output (depends on VS config)  
|   '-- Debug/ or Release/  
|       |-- NeedForSpeedInfinity.exe # The executable game  
|       |-- freeglut.dll             # Runtime DLLs needed  
|       |-- glew32.dll               # Runtime DLLs needed  
|       '-- highscore.txt            # Read/Written here at runtime  
|-- NeedForSpeedInfinity.rc          # Windows resource file (optional, for icons etc.)
```

Listing 10: Project Directory Structure

20. Complete Source Code Access

The full source code for this project, including all assets and build files, is available on GitHub:

<https://github.com/prabeshAryal/AwesomeCarGame.git>

21. Build and Run Instructions

The project is configured for compilation using Microsoft Visual Studio 2022.

1. Clone the repository from the GitHub link above.
2. Open the `NeedForSpeedInfinity.sln` file in Visual Studio 2022.
3. Ensure the solution configuration is set to `Debug` (or `Release`) and the platform is set to `x64`.
4. Make sure the necessary DLL files (`freeglut.dll`, `glew32.dll`) are either in the system path or placed in the output directory (`x64/Debug` or `x64/Release`) alongside the executable. The project settings might already configure this copy step.
5. Build the solution using `Build > Build Solution` (or press `F7`).
6. Run the game without debugging using `Debug > Start Without Debugging` (or press `Ctrl+F5`). Alternatively, navigate to the output directory (`x64/Debug` or `x64/Release`) and double-click `NeedForSpeedInfinity.exe`.

Game Controls

Action	Key(s)
Move Left	Left Arrow / A
Move Right	Right Arrow / D
Move Up	Up Arrow / W
Move Down	Down Arrow / S
Fire Projectile	Spacebar (During Gameplay)
Pause / Resume	P (During Gameplay)
Navigate Menu	Up/Down Arrow (In Menu)
Select Menu Item	Enter (In Menu)
Return to Menu	Spacebar (From Game Over Screen)
Exit Game	ESC (Any time)